A Mini Project Report Of

# PCC-201-COM : Data Structures

Submitted by

**Prathamesh Ganesh Bhute (48)**

Under the Guidance of
**Prof. Tejal H. Patil**




**Department of Computer Engineering**

**BHARATI VIDYAPEETH'S**
**COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115**

Savitribai Phule Pune University

**[2026-27]**

# BHARATI VIDYAPEETH'S
# COLLEGE OF ENGINEERING, LAVALE, PUNE

## CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Prathamesh Ganesh Bhute (48),** on **"Implementation of Job Scheduling [Graphs]"** is work done by her and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)

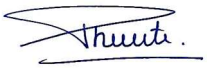
Dr. Lalit. V. Patil
Head of Computer Engg. Department

# ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Prof. Dr. Lalit. V. Patil,** Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.



**Prathamesh Ganesh Bhute**

# ABSTRACT

The mini project "**Implementation of Job Scheduling [Graphs]**" aims to demonstrate the practical application of graph data structures in scheduling jobs in the correct order. In many computer systems, some jobs must be completed before other jobs can start. These relationships are represented using a directed graph, where each node represents a job and each directed edge represents a dependency between jobs.

In this project, Python programming is used to implement job scheduling with the help of topological sorting. The program calculates the indegree of each job and uses a queue to select jobs whose dependencies have been completed. It then arranges the jobs in a valid execution order. If a circular dependency is detected, the program displays a message indicating that scheduling is not possible.

This project helps in understanding important data structure concepts such as graphs, vertices, directed edges, indegree, queues, and topological sorting. It also demonstrates how graph algorithms can be applied to real-world problems such as task management, project planning, and software development.

Overall, this mini project provides practical knowledge of graph implementation using Python and improves logical thinking, programming skills, and problem-solving abilities.

## TABLE OF CONTENTS


| Sr. No. | Contents | Page |
|---|---|---|
| 1 | Introduction | 1 |
| 2 | Literature Survey | 2 |
| 3 | Graph and Workflow Diagrams | 4 |
| 4 | System Requirement | 6 |
| 5 | System Design and Algorithm | 8 |
| 6 | System Code | 10 |
| 7 | Result | 14 |
| 8 | Application | 16 |
| 9 | Future Scope | 17 |
| 10 | Conclusion | 18 |
| 11 | References | 19 |


## LIST OF FIGURES


| Figure | Figure Description | Page No. |
|---|---|---|
| Fig 3.1 | UML Diagram | 04 |
| Fig 6.1 | System Code | 10 |
| Fig 6.2 | System Output | 12 |


## LIST OF TABLES


| Table No. | Table Name | Page No. |
|---|---|---|
| 1 | Literature Survey | 2 , 3 |

# 1. INTRODUCTION

## 1.1 Project Title :-

Job Scheduling [Graphs]

## 1.2 Introduction :-

Job scheduling is the process of arranging different jobs in a proper order so that they can be completed efficiently. In many computer systems and real-life situations, some jobs must be completed before other jobs can start. Therefore, it is important to identify the correct order of execution.

The main purpose of this project is to implement a job scheduling system using the Graph data structure in Python. A graph consists of vertices and edges. In this project, each vertex represents a job, and each directed edge represents a dependency between two jobs. For example, if Job B depends on Job A, then Job A must be completed before Job B starts.

This project uses topological sorting to arrange jobs in a valid execution order. The program calculates the indegree of each job, which represents the number of incoming edges to that job. A queue is used to select jobs whose dependencies have been completed. The selected jobs are added to the final scheduling order.

The program also checks whether a circular dependency exists between jobs. If a cycle is detected in the graph, it indicates that all jobs cannot be arranged in a valid order. The program then displays an appropriate message.

This project provides a simple understanding of graph representation, directed edges, indegree, queues, and topological sorting. It demonstrates how graph algorithms can be used to solve scheduling problems in computer systems, project planning, and task management.

The main aim of this project is to develop a simple and understandable Python program that demonstrates how jobs can be scheduled according to their dependencies. Through this implementation, students can improve their understanding of graph algorithms, logical problem-solving, and the practical use of data structures in real-life situations.

## 1.3 Objectives of the Project :-

The main objective of this project is to implement and demonstrate the practical use of graph data structures through a simple job scheduling system. The project aims to help students understand how jobs can be arranged in the correct order according to their dependencies using Python programming.

The specific objectives are as follows:

1. To understand the basic concepts of graphs, including vertices, directed edges, and indegree.
2. To represent different jobs and their dependencies using a directed graph.
3. To arrange jobs in the correct execution order using topological sorting.
4. To use a queue to process jobs whose dependencies have been completed.
6. To identify circular dependencies that prevent jobs from being scheduled in a valid order.
7. To improve logical thinking and problem-solving skills by implementing graph algorithms in Python.
8. To understand the practical applications of job scheduling in project planning, task management, and software development.

## 1.4 Scope :

The scope of this project is to develop a simple job scheduling system using graph data structures and Python programming. The system focuses on arranging jobs in the correct order based on their dependencies.

The scope of the project includes the following points:

1. **Job Representation**: Each job is represented as a vertex in a graph, making it easy to understand the different jobs in the system.

**2. Job Dependencies**: Directed edges are used to show the relationships between jobs and identify which job must be completed first.

**3**. **Job Scheduling**: Topological sorting is used to arrange jobs in a valid execution order according to their dependencies.

**4**. **Queue Operations**: A queue is used to process jobs that have no pending dependencies and are ready for execution.

**5**. **Python Implementation**: The project uses simple Python code to implement graph concepts and understand how the scheduling process works.

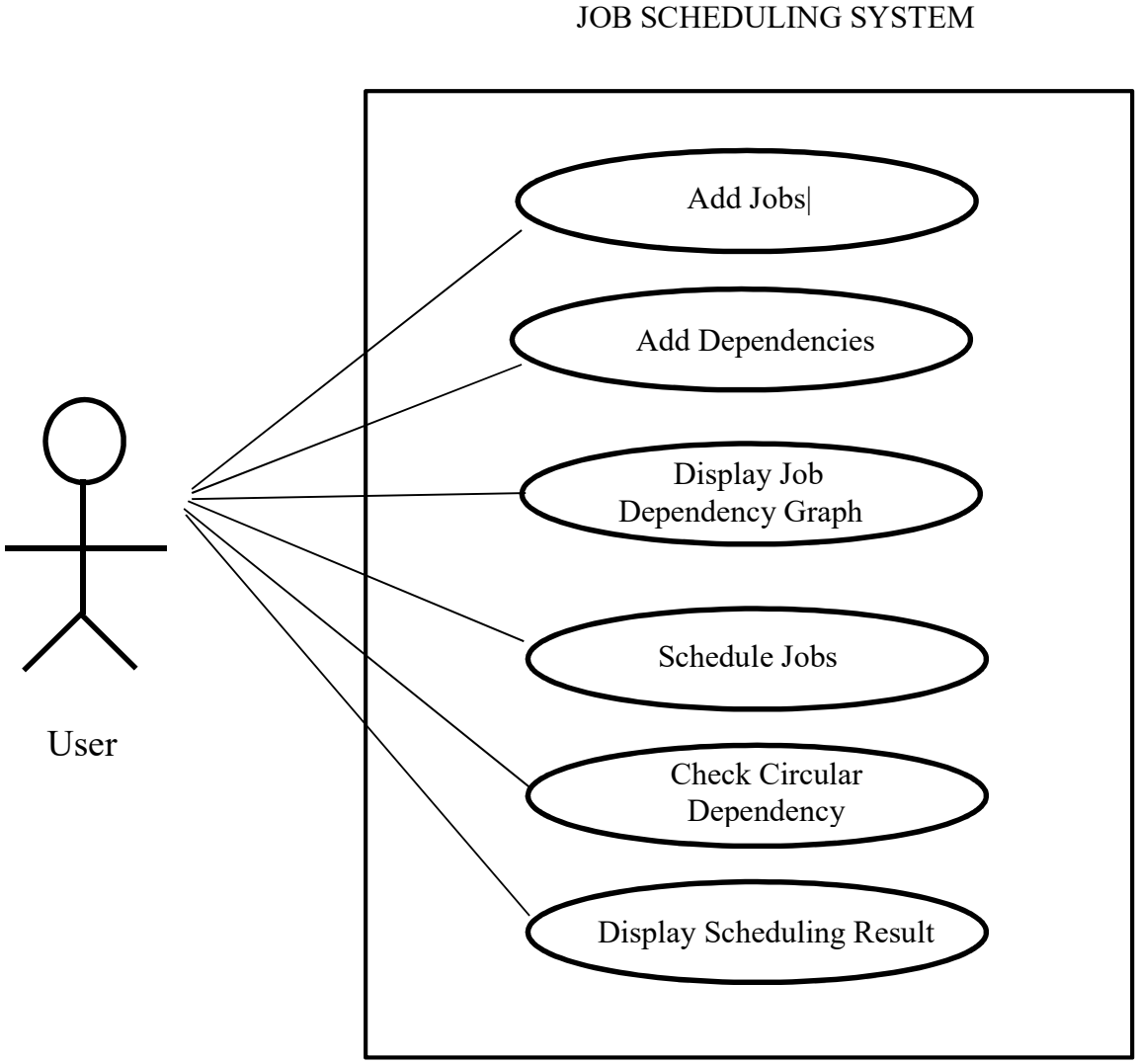
**6**. **Real-Life Applications**: The project concepts can be applied to task management, project planning, software development, and other activities where jobs must be completed in a specific order.

# 2. LITERATURE SURVEY

The Job Scheduling project demonstrates the practical application of graph data structures in arranging jobs according to their dependencies. Graphs are used to represent relationships between different jobs, while topological sorting helps determine the correct order of execution. This literature survey provides information about books and documentation related to graph data structures, graph algorithms, and Python programming.

| Year | Paper Name / Reference | Author | Conclusion |
|---|---|---|---|
| 2009 | Introduction to Algorithms, 3rd Edition | Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein | Explains graph algorithms, including topological sorting, which helps arrange jobs according to their dependencies. |
| 2013 | Data Structures and Algorithms in Python | Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser | Explains graph representations and algorithms useful for understanding relationships and dependencies between jobs. |
| 2011 | Algorithms, 4th Edition | Robert Sedgewick and Kevin Wayne | Explains important algorithms and graph-related concepts that help solve computational problems. |
| Current documentation | Python Documentation | Python Software Foundation | Provides information about Python lists and other programming features useful for implementing queues and job scheduling logic. |

# 3. UML DIAGRAMS



# 4. SYSTEM REQUIREMENT

## 4.1 HARDWARE REQUIREMENTS :-

1. **Processor Type**: Intel Core i3 or equivalent processor.
2. **Recommended Processor**: Intel Core i3 or higher.
3. **RAM**: Minimum 2 GB (4 GB recommended).
4. **Processor Speed**: 2.30 GHz or higher recommended.
5. **Hard Disk**: At least 100 MB of free space for Python, source code, and project files.
6. **Input Devices**: Keyboard and mouse.
7. **Output Device**: Monitor or laptop display.

## 4.2 SOFTWARE REQUIREMENTS :-

1. **Programming Language**: Python 3.8 or above.
2. **IDE / Code Editor**: Jupyter Notebook.
3. **Libraries Used**: Python standard library only. No external libraries are required.
4. **Operating System**: Windows 10/11, Linux, or macOS.

[ The program can be executed from a terminal using `python job_scheduling.py`. No internet connection is required after Python is installed.]

# 5. SYSTEM DESIGN AND ALGORITHM

## 5.1 Existing System :-

In the existing manual system, jobs are arranged according to their dependencies by a person. The person must identify which job should be completed first and which jobs depend on other jobs.

The manual process can create confusion when there are many jobs and dependencies. It may also be difficult to identify circular dependencies, where jobs depend on each other in a cycle.

* Limitations of the Existing System:

1. Jobs are arranged manually.
2. Finding the correct order can take more time when many jobs are involved.
3. Dependencies may be missed or entered incorrectly.
4. Identifying circular dependencies is difficult.

## 5.2 Proposed System :-

The proposed system uses a graph to represent jobs and their dependencies. Each job is represented as a vertex, and each directed edge represents a dependency between two jobs. The user enters the project title, job names, and dependency pairs. Kahn's Topological Sort algorithm is then used to calculate a valid job scheduling order. The system also detects circular dependencies and displays a suitable message if all jobs cannot be scheduled.

## 5.3 Algorithm: Kahn's Topological Sort :-

Kahn's Topological Sort is a graph algorithm used to arrange jobs in an order that follows their dependencies.

Steps of the algorithm:

1. Create a directed graph containing all jobs and their dependencies.
2. Calculate the indegree of every job. Indegree is the number of incoming edges to that job.
3. Insert all jobs with indegree zero into a queue. These jobs have no pending dependencies.
4. Remove one job from the queue and add it to the scheduling order.
5. Visit all dependent jobs of the selected job and decrease their indegree by one.
6. If any dependent job reaches indegree zero, insert it into the queue.
7. Repeat the process until the queue becomes empty.
8. If all jobs have been scheduled, display the correct scheduling order.
9. If some jobs remain unscheduled, report that a circular dependency exists.

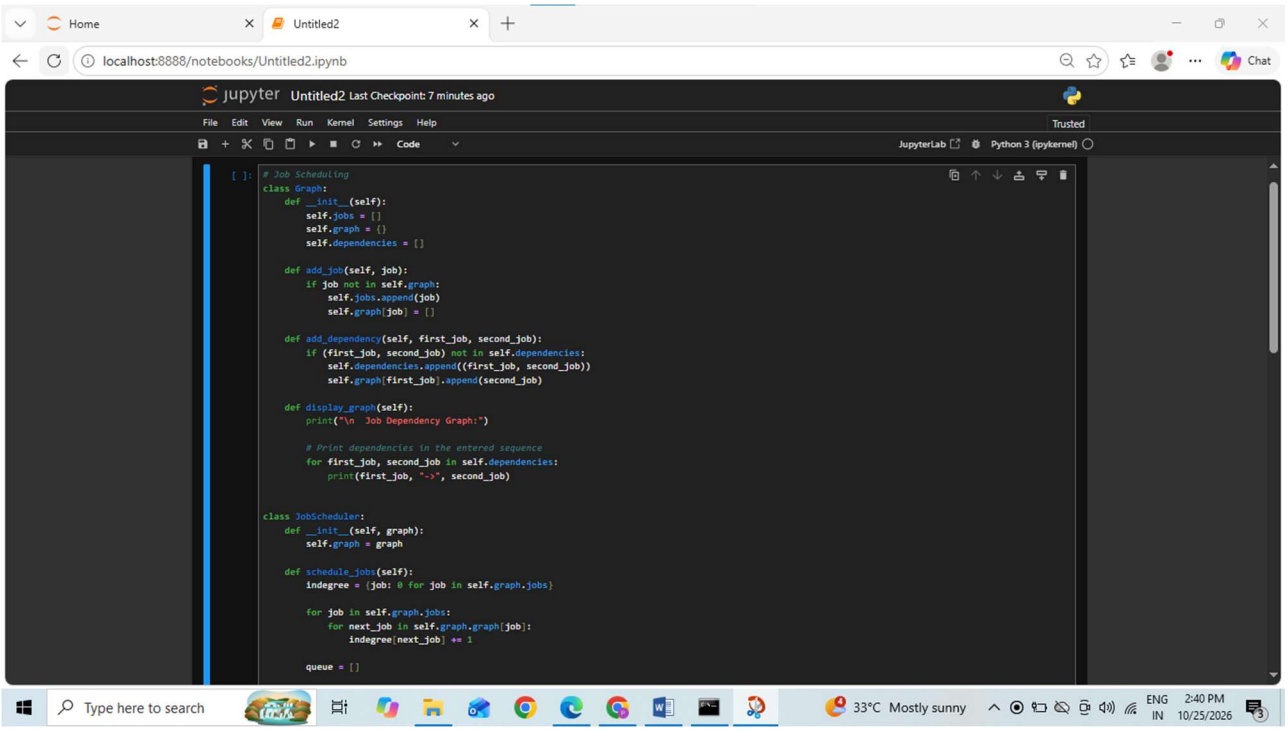
**5.5 Complexity Analysis :-**

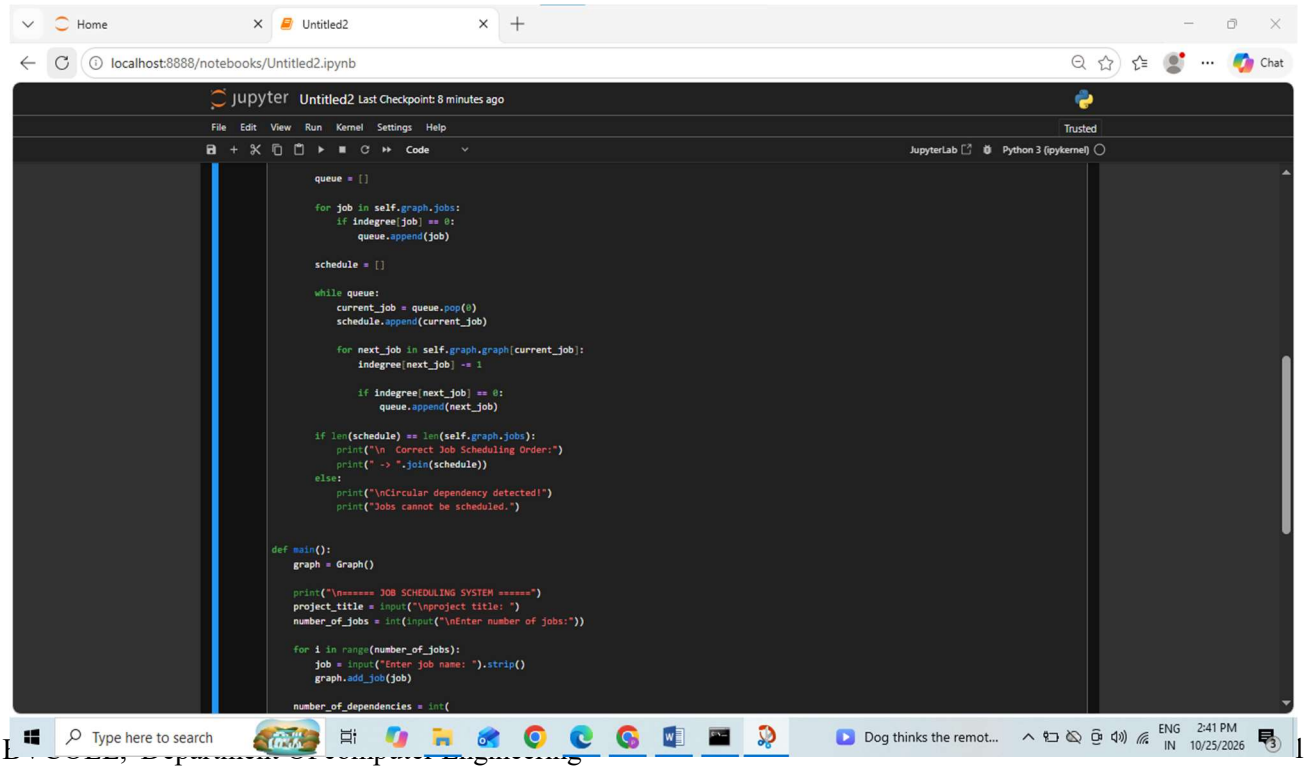
Complexity analysis describes how the algorithm's running time and memory requirements grow as the number of jobs and dependencies increases.

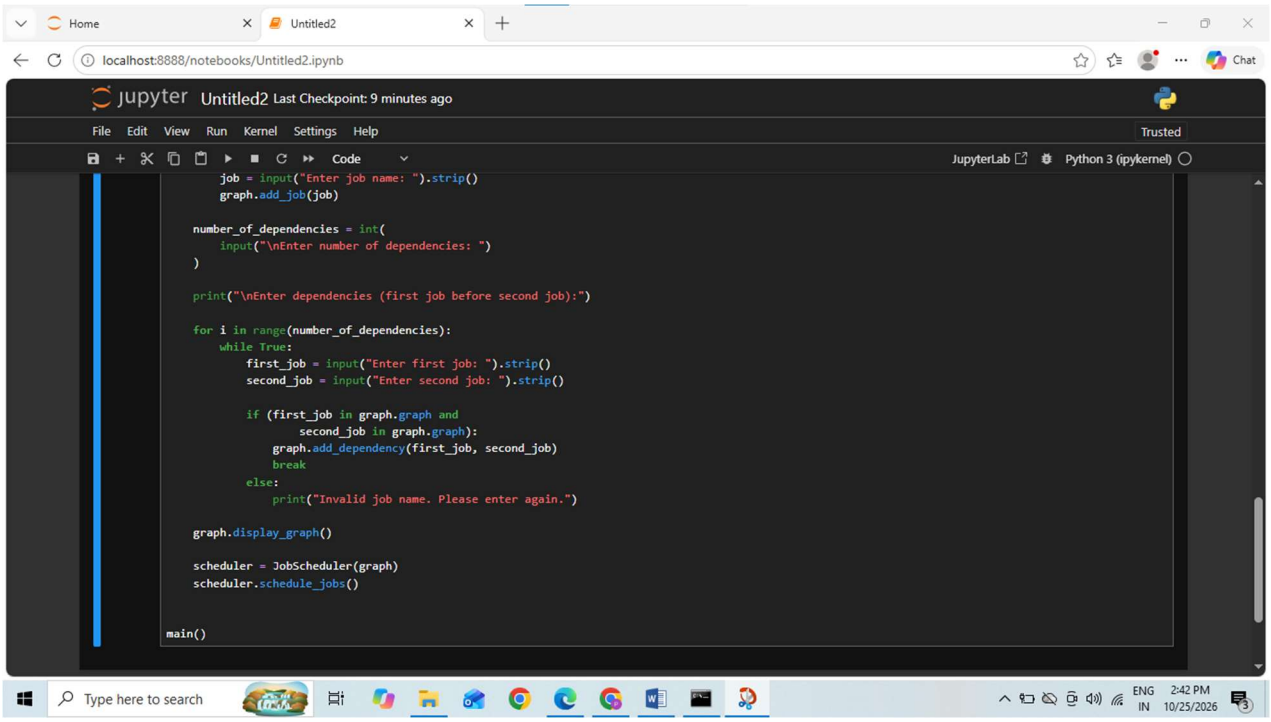
| Operation | Complexity | Explanation |
|---|---|---|
| Graph Construction | O(V+E) | Jobs and dependencies are stored in the graph. |
| Topological Sorting | $O(V^2+E)$ | The list-based queue uses `pop(0)`, which shifts remaining elements. |
| Overall Time Complexity | $O(V^2+E)$ | The queue operations increase the worst-case running time. |
| Space Complexity | O(V+E) | Memory is used for the graph, indegree dictionary, queue, and scheduling order. |

# 6. SYSTEM CODE

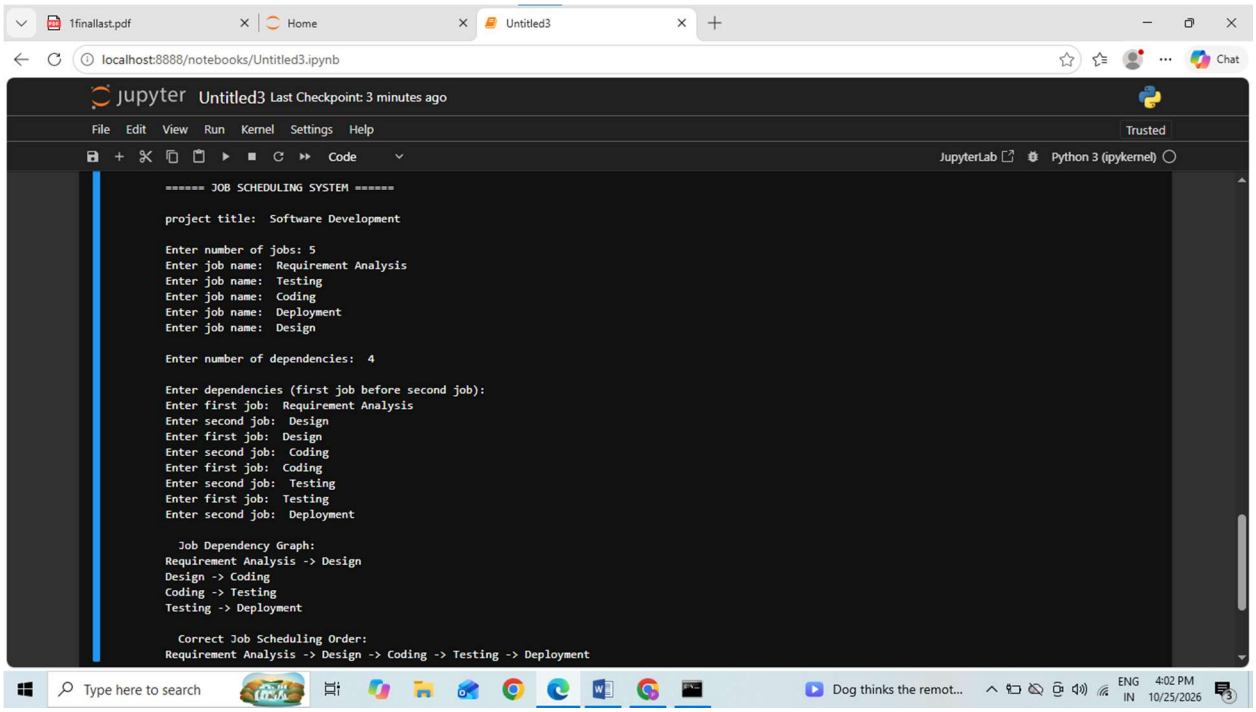
Save the following program as `job_scheduling.py`. The program uses only Python's standard library and validates that dependency endpoints refer to known jobs.




Home
Untitled2
localhost:8888/notebooks/Untitled2.ipynb
Chat
jupyter Untitled2 Last Checkpoint: 9 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
job = input("Enter job name: ").strip()
graph.add_job(job)
number_of_dependencies = int(
input("\nEnter number of dependencies: ")
)
print("\nEnter dependencies (first job before second job):")
for i in range(number_of_dependencies):
while True:
first_job = input("Enter first job: ").strip()
second_job = input("Enter second job: ").strip()
if (first_job in graph.graph and
second_job in graph.graph):
graph.add_dependency(first_job, second_job)
break
else:
print("Invalid job name. Please enter again.")
graph.display_graph()
scheduler = JobScheduler(graph)
scheduler.schedule_jobs()
main()
Type here to search
ENG
IN
2:42 PM
10/25/2026

# OUTPUT


1finallast.pdf
Home
Untitled3
localhost:8888/notebooks/Untitled3.ipynb
Chat
jupyter Untitled3 Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
====== JOB SCHEDULING SYSTEM ======
project title: Software Development
Enter number of jobs: 5
Enter job name: Requirement Analysis
Enter job name: Testing
Enter job name: Coding
Enter job name: Deployment
Enter job name: Design
Enter number of dependencies: 4
Enter dependencies (first job before second job):
Enter first job: Requirement Analysis
Enter second job: Design
Enter first job: Design
Enter second job: Coding
Enter first job: Coding
Enter second job: Testing
Enter first job: Testing
Enter second job: Deployment
Job Dependency Graph:
Requirement Analysis -> Design
Design -> Coding
Coding -> Testing
Testing -> Deployment
Correct Job Scheduling Order:
Requirement Analysis -> Design -> Coding -> Testing -> Deployment
Type here to search
Dog thinks the remot...
ENG
IN
4:02 PM
10/25/2026

# 7. TEST CASES AND RESULTS

The program is tested using valid job dependencies and circular dependencies. The test cases check whether the system generates a correct job scheduling order and detects circular dependencies. The expected results are shown below.

**7.1 TEST CASES :-**

| Test Case | Input / Condition | Expected Result | Status |
|---|---|---|---|
| TC1 | Valid job dependencies | Displays the correct scheduling order | Pass |
| TC2 | Single job without dependencies | Schedules the job successfully | Pass |
| TC3 | Invalid job name | Asks the user to enter the job name again | Pass |
| TC4 | Circular dependency between jobs | Displays a circular dependency error | Pass |

**Result**: The implementation generates a topological order for acyclic dependency graphs and reports a cycle when no valid order exists. The exact order may vary when multiple jobs are ready at the same time; any order that respects all dependencies is correct.

# 8. APPLICATIONS

The Job Scheduling System using Graphs has the following applications:

1. **Project Management**: Arranges project tasks in the correct order.
2. **Software Development**: Schedules coding, testing, and deployment tasks.
3. **Operating Systems**: Helps represent dependencies between tasks and processes.
4. **Build Systems:** Executes software build tasks according to their dependencies.
5. **Task Automation**: Runs automated tasks in the required order.
6. **Workflow Management**: Organizes activities that depend on completing other activities first.

# 9. FUTURE SCOPE

The Job Scheduling System can be improved in the future in the following ways:

1. **Graphical User Interface**: Develop a simple interface to make the system easier to use.
2. **Priority-Based Scheduling**: Schedule important jobs according to their priority.
3. **Time Estimation**: Add the estimated time required to complete each job.
4. **Database Integration**: Store job details and dependencies permanently.
5. **Cycle Improvement**: Provide suggestions to help users resolve circular dependencies.
6. **Real-Time Scheduling**: Update the job schedule whenever new jobs or dependencies are added.

# 10. CONCLUSION

The Job Scheduling System using Graphs is developed to manage jobs and their dependencies in the correct order. This project uses the graph data structure to represent jobs and the relationships between them. Each dependency defines which job must be completed before another job can begin.

The system uses Kahn's Topological Sorting Algorithm to generate a valid job scheduling order. It calculates the incoming dependencies of each job and processes jobs that have no pending dependencies. If a circular dependency is detected, the system displays an error message and informs the user that the jobs cannot be scheduled.

This project helps us understand important data structure concepts such as graphs, vertices, edges, indegree, queues, and topological sorting. It also demonstrates how graph algorithms can solve practical problems in project management, software development, and task scheduling.

# 11. REFERENCE


1. Python Software Foundation. Python 3 Documentation: Data Structures. https://docs.python.org/3/tutorial/datastructures
2. Kahn, A. B. (1962). “Topological Sorting of Large Networks.” Communications of the ACM, 5(11), 558–562. https://doi.org/10.1145/368996.369025.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
4. Python Software Foundation. Python 3 Documentation: collections — Container Datatypes. https://docs.python.org/3/library/collections.html#collections.deque.
5. Bharati Vidyapeeth’s College of Engineering, Lavale, Pune. Course Material for PCC-201-COM: Data Structures.